

Nix: построение воспроизводимых программных окружений на основе функционального программирования

Артём Белоусов

02.07.2026

Демо: Refal + Nix = ❤️

- Репозиторий с примером:
<https://git.flygrounder.ru/flygrounder/nix-refal-example>
- Nix пакет refal5:
<https://git.flygrounder.ru/flygrounder/refal5-flake.git>

Определения

- Программное окружение — набор программ, переменных окружения и настроек для выполнения вычислений в определённых условиях.
- Воспроизводимость окружения — сложность получения функционально эквивалентного окружения на другом устройстве.

Что не так с Docker?

Недостатки Docker:

- Дублирование библиотек
 - Занимает больше места
 - Сложнее следить за безопасностью
- При локальной разработке монтирование кода в контейнер создаёт сложности с правами

- От порядка слоёв зависит возможность кэширования и скорость сборки

```
COPY requirements.txt .
```

```
RUN pip install -r requirements.txt
```

```
COPY src .
```

- Чтобы в образ не попадали лишние данные приходится делать шаманства вроде

```
RUN apt-get update && \  
    apt-get install -y <package-name> && \  
    rm -rf /var/lib/apt/lists/*
```

Пакетный менеджер Nix

- Создан в 2003 году Eelco Dolstra в рамках PhD диссертации
- Собирает и устанавливает компоненты независимо друг от друга, что исключает незадекларированные зависимости
- Даёт гибкость сборки из исходников с возможностью кэширования результата
- Позволяет описывать композицию программных компонентов на чистом функциональном языке

Nixpkgs: репозиторий пакетов

- Содержит описание более 100 000 пакетов: инструменты для разработки, cli утилиты, gui приложения, библиотеки
- Предоставляет две модели обновлений
 - Стабильные релизы раз в полгода, например 26.05
 - unstable — самые свежие версии всех пакетов
- Коммит nixpkgs фиксирует версии всех пакетов

Пример среды разработки

<pre># shell.nix</pre>	<pre>~/Projects/nix-shell</pre>
<pre>{</pre>	<pre>> hello</pre>
<pre> pkgs ? import <nixpkgs> {},</pre>	<pre>fish: Unknown command: hello</pre>
<pre>}: pkgs.mkShellNoCC {</pre>	<pre>~/Projects/nix-shell</pre>
<pre> packages = [</pre>	<pre>> nix-shell --run fish</pre>
<pre> pkgs.hello</pre>	
<pre>];</pre>	<pre>~/Projects/nix-shell via *</pre>
<pre>}</pre>	<pre>impure (nix-shell)</pre>
	<pre>> hello</pre>
	<pre>Здравствуй, мир!</pre>

Как Nix создаёт окружения

Общая схема: Nix выражение $\rightarrow$ Деривация $\rightarrow$ Выход

- Деривация — стандартное описание сборки выходного пути.
Можно получить с помощью `nix-instantiate shell.nix`
- Выход — результат сборки деривации в изолированной среде.
Можно получить из деривации с помощью
`nix-store --realize /nix/store/<hash>-nix-shell.drv`
- Из Nix выражения можно сразу получить выход командой
`nix-build shell.nix`

Деривация

Пример сокращённого результата `nix-instantiate shell.nix`:

```
Derive([("out", "/nix/store/...-nix-shell", "", "")], [("/nix/store/...-hello-2.12.3.drv", ["out"]), ...], ["/nix/store/...-source-stdenv.sh", "/nix/store/...-default-builder.sh"], "x86_64-linux", "/nix/store/...-bash-5.3p9/bin/bash", ["-e", "/nix/store/l622p70vy8k5sh7y5wizi5f2mic6ynpg-source-stdenv.sh", "/nix/store/shkw4qm9qcw5sc5n1k5jznc83ny02r39-default-builder.sh"], ...)
```

Выход

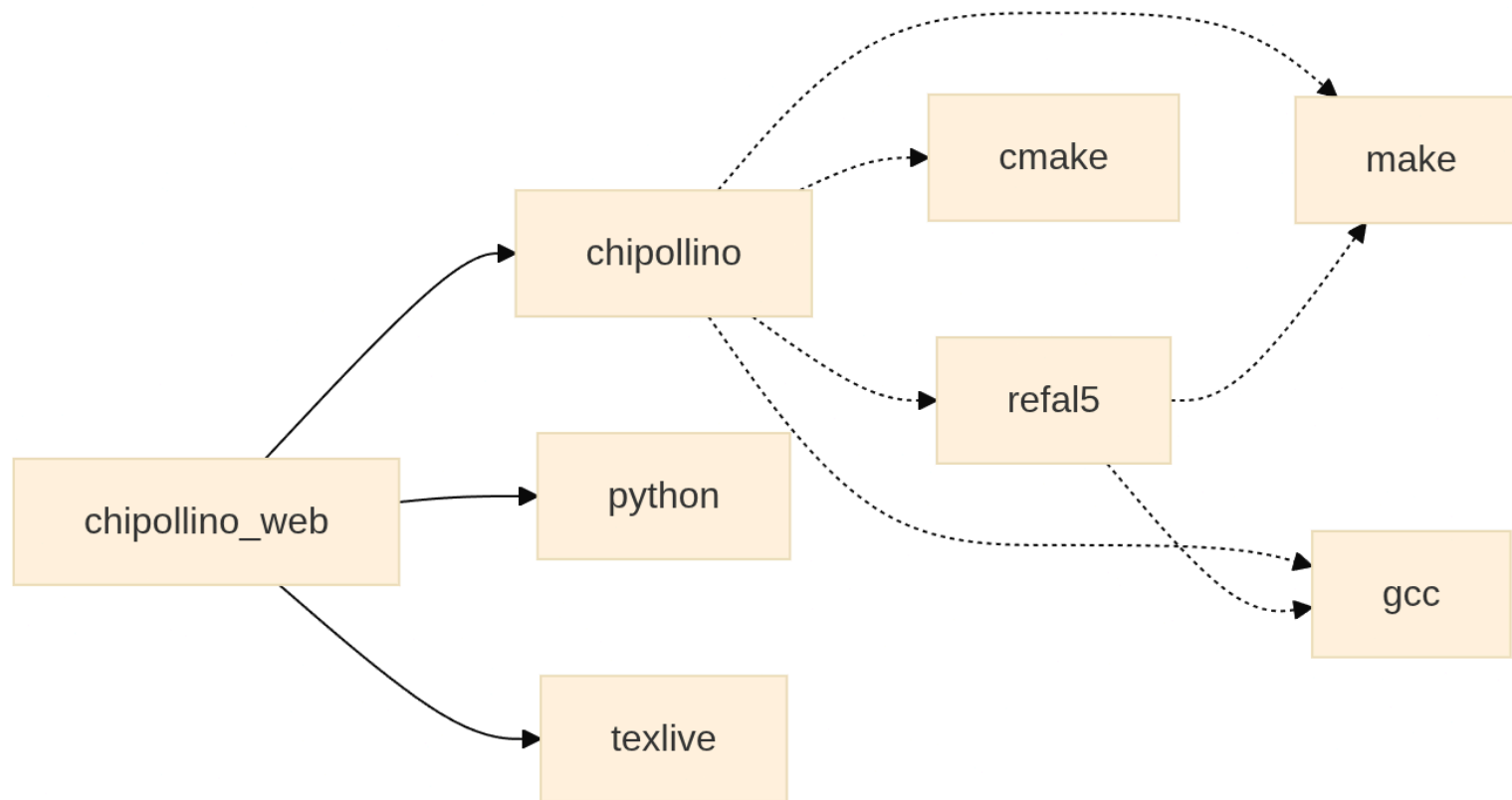
Результат выполнения `nix-store --realize /nix/store/...-nix-shell.drv`:

```
declare -x PATH="/nix/store/...hello-2.12.3/bin"  
# ...
```

Кэширование

- Выход создаётся из деривации в изолированной среде, поэтому результат можно кэшировать
- Официальный кэш `nixpkgs` — `cache.nixos.org`
- Можно поднять собственный кэш для своих дериваций
- Иногда разработчики программ предоставляют публичные кэши

chipollino_web: диаграмма зависимостей



chipollino_web: было

- `requirements.txt` без конкретных версий библиотек
- Инструкция по сборке Chipollino в другом репозитории
- Docker образ с зависимостями из репозитория debian

chipollino_web: стало

- `devenv.nix` - описание среды разработки и контейнеров
- `devenv.yaml` - список всех зависимостей
- `devenv.lock` - фиксирует конкретные версии всех зависимостей

Советы по использованию Nix

- Фиксируйте конкретный коммит nixpkgs
- Периодически выполняйте сборку мусора:
`nix-collect-garbage -d`
- Указывайте лимиты при сборке:
`nix-build docker.nix --max-jobs 1 --cores 1`

Проекты в экосистеме Nix

- devenv — удобная обёртка над Nix для создания окружений для разработки
- NixOS — Linux дистрибутив, в котором конфигурация всей системы описывается как Nix выражение
- Home Manager — модули для управления пользовательским окружением

Заключение

1. Узнали об основах управления окружениями с помощью Nix
2. Рассмотрели практический пример настройки окружения через Nix

Анонс



@BOTIK_NIX